

# Advanced Manual Smart Contract Audit

May 28, 2026

 [CoinsultAudits](#)

 [t.me/coinsult\\_tg](https://t.me/coinsult_tg)

 [coinsult.net](https://coinsult.net)

Audit requested by



**Kancle Coin**

**FiDNTyuVE3V89DXRyoEWj1Gv4bycDL9Hub9tGmoHpump**

# Global Overview

## Manual Code Review

In this audit report we will highlight the following issues:

| Vulnerability Level | Total | Pending | Acknowledged | Resolved |
|---------------------|-------|---------|--------------|----------|
| ● Informational     | 0     | 0       | 0            | 0        |
| ● Low-Risk          | 0     | 0       | 0            | 0        |
| ● Medium-Risk       | 0     | 0       | 0            | 0        |
| ● High-Risk         | 0     | 0       | 0            | 0        |

## Centralization Risks

Coinsult checked the following privileges:

| Contract Privilege             | Description                               |
|--------------------------------|-------------------------------------------|
| Owner needs to enable trading? | ● Owner does not need to enable trading   |
| Owner can mint?                | ● Owner cannot mint new tokens            |
| Owner can blacklist?           | ● Owner cannot blacklist addresses        |
| Owner can set fees?            | ● Owner can set the sell fee to 0%        |
| Owner can exclude from fees?   | ● Owner cannot exclude from fees          |
| Can be honeypotted?            | ● Owner cannot pause the contract         |
| Owner can set Max TX amount?   | ● Owner cannot set max transaction amount |

More owner privileges are listed later in the report.

# Table of Contents

## 1. Audit Summary

- 1.1 Audit scope
- 1.2 Tokenomics
- 1.3 Source Code

## 2. Disclaimer

## 3. Global Overview

- 3.1 Informational issues
- 3.2 Low-risk issues
- 3.3 Medium-risk issues
- 3.4 High-risk issues

## 4. Vulnerabilities Findings

## 5. Contract Privileges

- 5.1 Maximum Fee Limit Check
- 5.2 Contract Pausability Check
- 5.3 Max Transaction Amount Check
- 5.4 Exclude From Fees Check
- 5.5 Ability to Mint Check
- 5.6 Ability to Blacklist Check
- 5.7 Owner Privileges Check

## 6. Notes

- 6.1 Notes by Coinsult
- 6.2 Notes by Kancler Coin

## 7. Contract Snapshot

## 8. Website Review

## 9. Certificate of Proof

# Audit Summary

|                         |                                                       |
|-------------------------|-------------------------------------------------------|
| Project Name            | Kancler Coin                                          |
| Website                 | <a href="https://kncler.com/">https://kncler.com/</a> |
| Blockchain              | Solana                                                |
| Smart Contract Language | Rust                                                  |
| Contract Address        | FiDNTyuVE3V89DXRyoEWj1Gv4bycDL9Hub9tGmoHpump          |
| Audit Method            | Static Analysis, Manual Review                        |
| Date of Audit           | 28 May 2026                                           |

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

# Audit Scope

Coinsult was commissioned by Kancler Coin to perform an audit based on the following code:

<https://solscan.io/token/FiDNTyuVE3V89DXRyoEWj1Gv4bycDL9Hub9tGmoHpump>

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

## Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

### Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

### Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

### Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

# Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

| Vulnerability Level | Description                                                      |
|---------------------|------------------------------------------------------------------|
| ● Informational     | Does not compromise the functionality of the contract in any way |
| ● Low-Risk          | Won't cause any problems, but can be adjusted for improvement    |
| ● Medium-Risk       | Will likely cause problems and it is recommended to adjust       |
| ● High-Risk         | Will definitely cause problems, this needs to be adjusted        |

Coinsult has four statuses that are used for each risk level. Below we explain them briefly.

| Risk Status  | Description                                              |
|--------------|----------------------------------------------------------|
| Total        | Total amount of issues within this category              |
| Pending      | Risks that have yet to be addressed by the team          |
| Acknowledged | The team is aware of the risks but does not resolve them |
| Resolved     | The team has resolved and remedied the risk              |

# SWC Attack Analysis

The Smart Contract Weakness Classification Registry (SWC Registry) is an implementation of the weakness classification scheme proposed in EIP-1470. It is loosely aligned to the terminologies and structure used in the Common Weakness Enumeration (CWE) while overlaying a wide range of weakness variants that are specific to smart contracts.

| ID      | Description                          | Status |
|---------|--------------------------------------|--------|
| SWC-100 | Function Default Visibility          | Passed |
| SWC-101 | Integer Overflow and Underflow       | Passed |
| SWC-102 | Outdated Compiler Version            | Passed |
| SWC-103 | Floating Pragma                      | Passed |
| SWC-104 | Unchecked Call Return Value          | Passed |
| SWC-105 | Unprotected Ether Withdrawal         | Passed |
| SWC-106 | Unprotected SELFDESTRUCT Instruction | Passed |
| SWC-107 | Reentrancy                           | Passed |
| SWC-108 | State Variable Default Visibility    | Passed |
| SWC-109 | Uninitialized Storage Pointer        | Passed |
| SWC-110 | Assert Violation                     | Passed |
| SWC-111 | Use of Deprecated Solidity Functions | Passed |
| SWC-112 | Delegatecall to Untrusted Callee     | Passed |
| SWC-113 | DoS with Failed Call                 | Passed |
| SWC-114 | Transaction Order Dependence         | Passed |
| SWC-115 | Authorization through tx.origin      | Passed |

|         |                                                         |        |
|---------|---------------------------------------------------------|--------|
| SWC-116 | Block values as a proxy for time                        | Passed |
| SWC-117 | Signature Malleability                                  | Passed |
| SWC-118 | Incorrect Constructor Name                              | Passed |
| SWC-119 | Shadowing State Variables                               | Passed |
| SWC-120 | Weak Sources of Randomness from Chain Attributes        | Passed |
| SWC-121 | Missing Protection against Signature Replay Attacks     | Passed |
| SWC-122 | Lack of Proper Signature Verification                   | Passed |
| SWC-123 | Requirement Violation                                   | Passed |
| SWC-124 | Write to Arbitrary Storage Location                     | Passed |
| SWC-125 | Incorrect Inheritance Order                             | Passed |
| SWC-126 | Insufficient Gas Griefing                               | Passed |
| SWC-127 | Arbitrary Jump with Function Type Variable              | Passed |
| SWC-128 | DoS With Block Gas Limit                                | Passed |
| SWC-129 | Typographical Error                                     | Passed |
| SWC-130 | Right-To-Left-Override control character (U+202E)       | Passed |
| SWC-131 | Presence of unused variables                            | Passed |
| SWC-132 | Unexpected Ether balance                                | Passed |
| SWC-133 | Hash Collisions With Multiple Variable Length Arguments | Passed |
| SWC-134 | Message call with hardcoded gas amount                  | Passed |
| SWC-135 | Code With No Effects                                    | Passed |
| SWC-136 | Unencrypted Private Data On-Chain                       | Passed |



## Maximum Fee Limit Check

| Error Code | Description                               |
|------------|-------------------------------------------|
| CEN-01     | Centralization: Operator Fee Manipulation |

Coinsult tests if the owner of the smart contract can set the transfer, buy or sell fee to 25% or more. It is bad practice to set the fees to 25% or more, because owners can prevent healthy trading or even stop trading when the fees are set too high.

| Type of fee      | Description |
|------------------|-------------|
| Max transfer fee | 0%          |
| Max buy fee      | 0%          |
| Max sell fee     | 0%          |

## Contract Honeypot Check

| Error Code | Description                          |
|------------|--------------------------------------|
| CEN-02     | Centralization: Operator Pausability |

Coinsult tests if the owner of the smart contract has the ability to pause the contract. If this is the case, users can no longer interact with the smart contract; users can no longer trade the token.

| Privilege Check               | Description                                                                                                       |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Can owner pause the contract? |  Owner cannot pause the contract |

## Max Transaction Amount Check

| Error Code | Description                                       |
|------------|---------------------------------------------------|
| CEN-03     | Centralization: Operator Transaction Manipulation |

Coinsult tests if the owner of the smart contract can set the maximum amount of a transaction. If the transaction exceeds this limit, the transaction will revert. Owners could prevent normal transactions to take place if they abuse this function.

| Privilege Check              | Description                                                                                                               |
|------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Can owner set max tx amount? |  Owner cannot set max transaction amount |

## Exclude From Fees Check

| Error Code | Description                        |
|------------|------------------------------------|
| CEN-04     | Centralization: Operator Exclusion |

Coinsult tests if the owner of the smart contract can exclude addresses from paying tax fees. If the owner of the smart contract can exclude from fees, they could set high tax fees and exclude themselves from fees and benefit from 0% trading fees. However, some smart contracts require this function to exclude routers, dex, cex or other contracts / wallets from fees.

| Privilege Check              | Description                                                                                                      |
|------------------------------|------------------------------------------------------------------------------------------------------------------|
| Can owner exclude from fees? |  Owner cannot exclude from fees |

## Ability To Mint Check

| Error Code | Description                              |
|------------|------------------------------------------|
| CEN-05     | Centralization: Operator Increase Supply |

Coinsult tests if the owner of the smart contract can mint new tokens. If the contract contains a mint function, we refer to the token's total supply as non-fixed, allowing the token owner to "mint" more tokens whenever they want.

A mint function in the smart contract allows minting tokens at a later stage. A method to disable minting can also be added to stop the minting process irreversibly.

Minting tokens is done by sending a transaction that creates new tokens inside of the token smart contract. With the help of the smart contract function, an unlimited number of tokens can be created without spending additional energy or money.

| Privilege Check | Description                                                                                                      |
|-----------------|------------------------------------------------------------------------------------------------------------------|
| Can owner mint? |  Owner cannot mint new tokens |

## Enable Trading

| Error Code | Description                             |
|------------|-----------------------------------------|
| CEN-06     | Centralization: Operator enable trading |

Coinsult tests if the owner of the smart contract needs to manually enable trading before everyone can buy & sell. If the owner needs to manually enable trading, this poses a high centralization risk.

If the owner needs to manually enable trading, make sure to check if the project has a SAFU badge or a trusted KYC badge. Always DYOR when investing in a project that needs to manually enable trading.

| Privilege Check                | Description                                                                                                             |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| Owner needs to enable trading? |  Owner does not have to enable trading |

## Ability To Blacklist Check

| Error Code | Description                                 |
|------------|---------------------------------------------|
| CEN-07     | Centralization: Operator Dissallows Wallets |

Coinsult tests if the owner of the smart contract can blacklist accounts from interacting with the smart contract. Blacklisting methods allow the contract owner to enter wallet addresses which are not allowed to interact with the smart contract.

This method can be abused by token owners to prevent certain / all holders from trading the token. However, blacklists might be good for tokens that want to rule out certain addresses from interacting with a smart contract.

| Privilege Check      | Description                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| Can owner blacklist? |  Owner cannot blacklist addresses |

## Other Owner Privileges Check

| Error Code | Description                         |
|------------|-------------------------------------|
| CEN-100    | Centralization: Operator Priviliges |

Coinsult lists all important contract methods which the owner can interact with.

✓ No other important owner privileges to mention.



# Notes

## Notes by Kancler Coin

No notes provided by the team.

## Notes by Coinsult

No notes provided by Coinsult

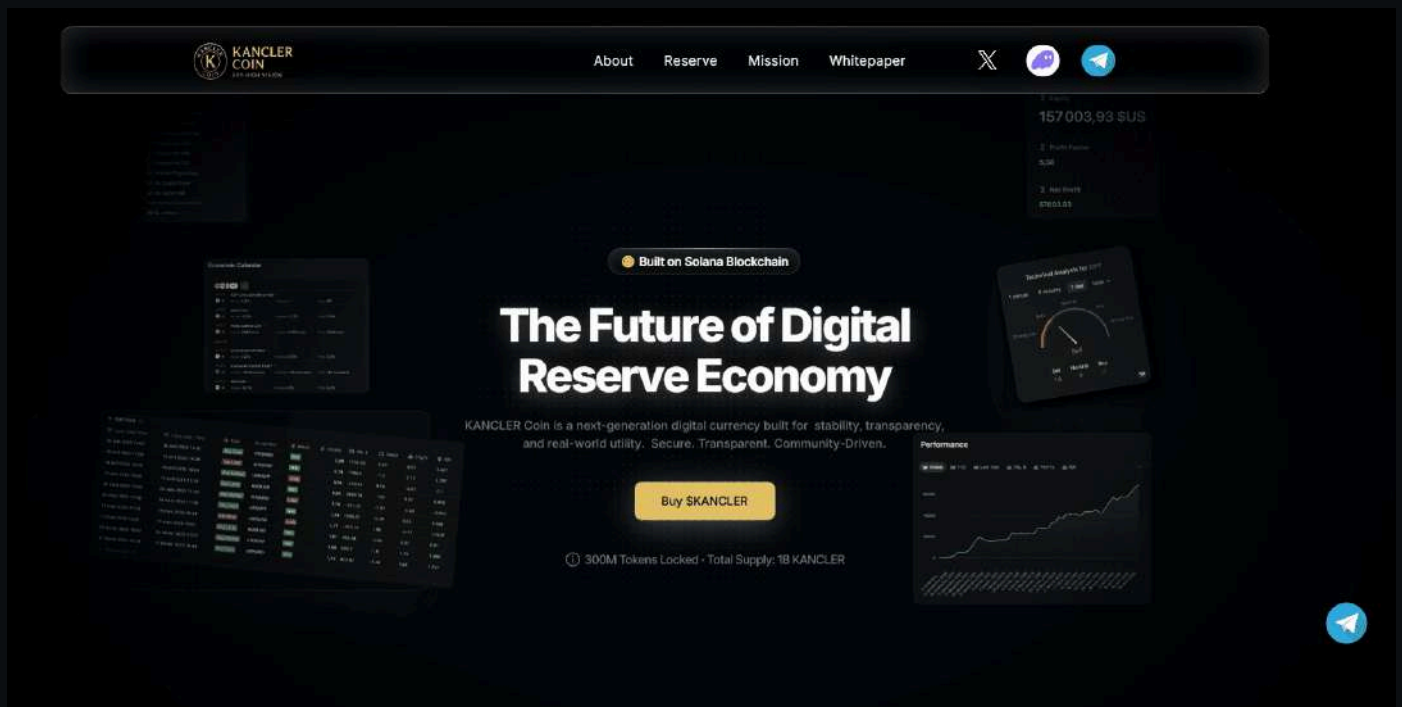
# Contract Snapshot

This is how the constructor of the contract looked at the time of auditing the smart contract.

```
{8 items
key:
4
updateAuthority:
"TSLvdd1pWpHVjahSpsvCXUbgwsL3JAcvokwaKt1eokM"
mint:
"FiDNTyuVE3V89DXRyoEWj1Gv4bycDL9Hub9tGmoHpump"
data:{5 items
name:
"kncler coin"
symbol:
"kancler"
uri:
"https://ipfs.io/ipfs/bafkreiamup6odybimlczy62b3tw6jo3jrjc3g3abz7g7tpqeursrfgme"
sellerFeeBasisPoints:
0
creators:[1 item
0:{3 items
address:
"JDweBK5HvVHJ61XTvRPtGFtvAiPWMYYhkn1ZyEA5qcnj"
verified:
0
share:
100
}
]
}
primarySaleHappened:
0
isMutable:
0
editionNonce:
254
tokenStandard:
2
}
```

# Website Review

Coinsult checks the website completely manually and looks for visual, technical and textual errors. We also look at the security, speed and accessibility of the website. In short, a complete check to see if the website meets the current standard of the web development industry.



| Type of check             | Description                                    |
|---------------------------|------------------------------------------------|
| Mobile friendly?          | ● The website is mobile friendly               |
| Contains jQuery errors?   | ● The website does not contain jQuery errors   |
| Is SSL secured?           | ● The website is SSL secured                   |
| Contains spelling errors? | ● The website does not contain spelling errors |

# Certificate of Proof

● Not KYC verified by Coinsult

## Kancler Coin

Audited by Coinsult.net



Date: 28 May 2026

✓ Advanced Manual Smart Contract Audit

# Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.

# End of report

## **Smart Contract Audit**

 CoinsultAudits

 info@coinsult.net

 coinsult.net

Request your smart contract audit / KYC

**t.me/coinsult\_tg**